

FUNDAÇÃO ESCOLA DE COMÉRCIO ÁLVARES PENTEADO - FECAP

Bacharelado em Ciência da Computação

Teoria da Computação e Linguagem Formal – Professora Kátia Milani Lara Bossi

Estudantes:

- André Gregório dos Santos – RA: 24026489
- Guilherme Reis Fogolin de Godoy – RA: 24026241
- Pedro Henrique Nascimento Lemos – RA: 23025380
- Yan Ramos Cezareto – RA: 24026005

Turma: 6NACOMP_S

NockHug: Modelagem formal do controle por voz de uma máquina lançadora de bolas de ping-pong

Apresentação do projeto

O NockHug é um projeto interdisciplinar desenvolvido em parceria com uma empresa do segmento de máquinas lançadoras de bolas de tênis. Nosso grupo construiu uma mini máquina lançadora de bolas de ping-pong cujo diferencial é o controle por voz: o usuário conecta o celular à máquina, abre uma aplicação web, ativa o microfone e inicia ou interrompe o disparo de bolas falando comandos como "liga" e "desliga".

A arquitetura do protótipo tem três camadas. O ESP32 opera como ponto de acesso Wi-Fi e servidor HTTP: é a rede dele que o celular acessa e é dele que a página web é servida. No navegador do celular, o reconhecimento de fala (Speech-to-Text do Google, por meio da Web Speech API, configurado para português do Brasil) converte a fala em texto, e um trecho de JavaScript verifica se o texto contém alguma palavra de comando. Se contiver, o navegador envia uma requisição HTTP ao ESP32 (/ligar ou /desligar), que aciona os dois motores por PWM, com rampa de aceleração ao ligar, ou os desliga de imediato.

Este documento formaliza parte do comportamento do sistema: o ciclo de controle por voz, desde a conexão do celular até o acionamento e a parada dos motores. Esse recorte é modelado por dois autômatos finitos que se comunicam: M1, que descreve a sessão de uso (conexão, escuta, motores ligados), e M2, que descreve o reconhecimento de um comando a partir do texto transcrito. São usados os conceitos de autômatos finitos, linguagens formais, expressões regulares e gramáticas

estudados na disciplina, e as seções de 1 a 8 correspondem, na ordem, aos oito itens exigidos da entrega (conforme documentação oficial).

O comportamento modelado não é uma abstração criada para a disciplina e, sim, corresponde ao que está implementado no código do protótipo (firmware do ESP32 e página web que ele serve), aqui descrito em termos matemáticos, isto é, conjunto de estados, alfabeto de eventos, função de transição, ações de saída, linguagem reconhecida, expressão regular equivalente e gramática geradora.

Índice

1 Identificação dos estados do sistema	3
2 Eventos (entradas)	5
3 Diagrama de estados	6
4 Tabela de transições	7
5 Linguagem do sistema	9
6 Expressão regular	11
7 Gramática	13
8 Justificativa	15
9 Reflexões	17
10 Conclusão	18
11 Anexo I	19

1 Identificação dos estados do sistema

O NockHug é atravessado por um fluxo de ponta a ponta: máquina ligada, celular conectado, microfone ativo, fala, texto transcrito, validação do comando, requisição ao ESP32 e motores acionados. Dois trechos desse fluxo têm comportamento genuinamente sequencial e por isso se prestam à modelagem por autômato finito: a sessão de uso, tratada em M1 (Seção 1.1), e o reconhecimento do comando dentro da frase, tratado em M2 (Seção 1.2). A transcrição da fala em si é feita por um serviço externo ao sistema e é tratada como uma caixa-preta cuja saída, o texto, é a entrada de M2.

1.1 M1 — Sessão de controle por voz

M1 é um autômato finito não determinístico com transição ϵ (AFN- ϵ) de cinco estados, formalmente $M_1 = (Q_1, \Sigma_1, \delta_1, R, F_1)$, com $Q_1 = \{R, C, E, A, M\}$, estado inicial R e $F_1 = \{R\}$. O estado R é ao mesmo tempo inicial e o único de aceitação: o sistema "aceita" uma sequência de eventos sempre que retorna ao repouso, o que representa uma sessão de uso completa. A transição ϵ , discutida nas seções 4.1 e 8.5, é o motivo dessa classificação.

Quadro 1 — Estados do autômato da sessão de controle por voz (M1).

Estado	Símbolo	O que representa
REPOUSO	R	Estado inicial e de aceitação. Máquina ligada, com a rede Wi-Fi do ESP32 ativa e o servidor HTTP aguardando conexões. Nenhum celular conectado; os pinos dos motores estão em zero (garantido no setup()).
CONECTADO	C	O celular está conectado à rede do ESP32 e a página foi carregada; ao carregar, ela consulta o estado dos motores (/status) e o exibe. Microfone inativo: nenhum comando pode ser dado por voz.
ESCUTANDO	E	Microfone ativo, com reconhecimento de voz contínuo em pt-BR (a escuta é reiniciada automaticamente quando o navegador a encerra). Motores parados; o sistema aguarda uma frase com comando.
ACELERANDO	A	Um comando de ligar foi validado e a requisição /ligar chegou ao ESP32, que executa a rampa de aceleração dos dois motores. Não aguarda evento externo: a saída ocorre sozinha, ao fim da rampa (transição ϵ).
MOTORES_LIGADOS	M	Rampa concluída; motores em regime, com disparo de bolas em andamento. A escuta continua ativa, à espera de um comando de desligar.

1.2 M2 — Reconhecedor de comandos

M2 é um autômato finito determinístico (AFD) de três estados, formalmente $M_2 = (Q_2, \Sigma_2, \delta_2, q_0, F_2)$, com $Q_2 = \{q_0, q_1, q_2\}$ e $F_2 = \{q_1, q_2\}$. Ele é executado no navegador e lê a frase transcrita palavra por palavra, guardando apenas o que já viu: nenhuma palavra-chave, uma palavra de ligar ou uma palavra de desligar. Aceitar significa que a frase contém um comando, e o estado em que a leitura termina diz qual. Rejeitar (terminar em q_0) significa "comando não mapeado": nada é enviado ao ESP32.

Quadro 2 — Estados do autômato reconhecedor de comandos (M2).

Estado	Símbolo	O que representa	Resultado ao fim da frase
NEUTRO	q_0	Estado inicial, não aceitador. Nenhuma palavra-chave foi lida até o momento.	Frase rejeitada; nenhuma requisição é enviada.
LIGAR	q_1	Estado de aceitação. Foi lida ao menos uma palavra de ligar e nenhuma de desligar.	Requisição /ligar (evento l de M1).
DESLIGAR	q_2	Estado de aceitação e absorvente: foi lida ao menos uma palavra de desligar e nada tira o autômato daqui, o que dá prioridade ao desligar.	Requisição /desligar (evento d de M1).

A ligação entre as duas máquinas é direta: o resultado de M2 é o símbolo l ou d que alimenta M1. Ligar a máquina, primeiro passo do fluxo, não é um evento, e sim a condição inicial de M1: o sistema começa em R.

2 Eventos (entradas)

Os eventos são as condições observáveis capazes de provocar mudança de estado. Como há dois autômatos, há dois alfabetos: um de eventos da sessão, para M1, e um de classes de palavras, para M2.

2.1 Eventos da sessão (M1)

O alfabeto de entrada de M1 é $\Sigma_1 = \{w, v, l, d, p, x\}$. Cada símbolo corresponde a uma ação do usuário ou a um comando já validado, e cada um tem uma condição observável no código.

Quadro 3 — Alfabeto Σ_1 e condição correspondente no sistema.

Símbolo	Evento	Condição no sistema
w	Conexão estabelecida	O celular conecta à rede Wi-Fi do ESP32 e a página é carregada (GET / seguido de GET /status).
v	Microfone ativado	O usuário toca em "Iniciar Escuta de Voz": a função toggleVoice() inicia o reconhecimento e isListening passa a verdadeiro.
l	Comando de ligar reconhecido	M2 aceita a frase em q_1 ; o navegador envia GET /ligar ao ESP32.
d	Comando de desligar reconhecido	M2 aceita a frase em q_2 ; o navegador envia GET /desligar ao ESP32.
p	Microfone desativado	O usuário toca novamente no botão de escuta: toggleVoice() encerra o reconhecimento e isListening passa a falso.
x	Desconexão	O celular deixa a rede do ESP32 ou a página é fechada: a sessão termina.

Vale destacar que l e d não são a fala bruta, e sim comandos já validados por M2. O que o usuário diz e como o Speech-to-Text o transcreve não pertence ao alfabeto de M1.

2.2 Classes de palavras (M2)

O alfabeto de M2 é $\Sigma_2 = \{L, D, N\}$. A frase transcrita é normalizada (espaços das pontas removidos e letras minúsculas) e, para efeito do modelo, vista como uma sequência de palavras, cada uma com uma classe conforme o termo-chave que contém.

Quadro 4 — Alfabeto Σ_2 e critério de classificação de cada palavra.

Símbolo	Classe da palavra	Critério no código
L	Palavra de ligar	Contém ligar, liga, acender ou on e não contém nenhum termo do grupo de desligar.
D	Palavra de desligar	Contém desligar, desliga, apagar ou off.
N	Palavra neutra	Qualquer outra palavra da frase.

Como nenhum termo-chave contém espaço, procurar o termo na frase inteira, como faz o código, equivale a classificar palavra por palavra e depois ler a sequência de classes. É isso que permite modelar a validação como um autômato sobre Σ_2 . A classificação testa primeiro o grupo de desligar porque "desligar" e "desliga" contém "ligar" e "liga": sem essa ordem, todo desligar seria lido como ligar.

3 Diagrama de estados

As Figuras 1 e 2 representam graficamente M1 e M2. Em ambas, os estados são círculos, os estados de aceitação têm círculo duplo (REPOUSO em M1; LIGAR e DESLIGAR em M2), as transições são setas rotuladas com o símbolo lido, ϵ indica uma transição espontânea e a seta isolada à esquerda indica o estado inicial.

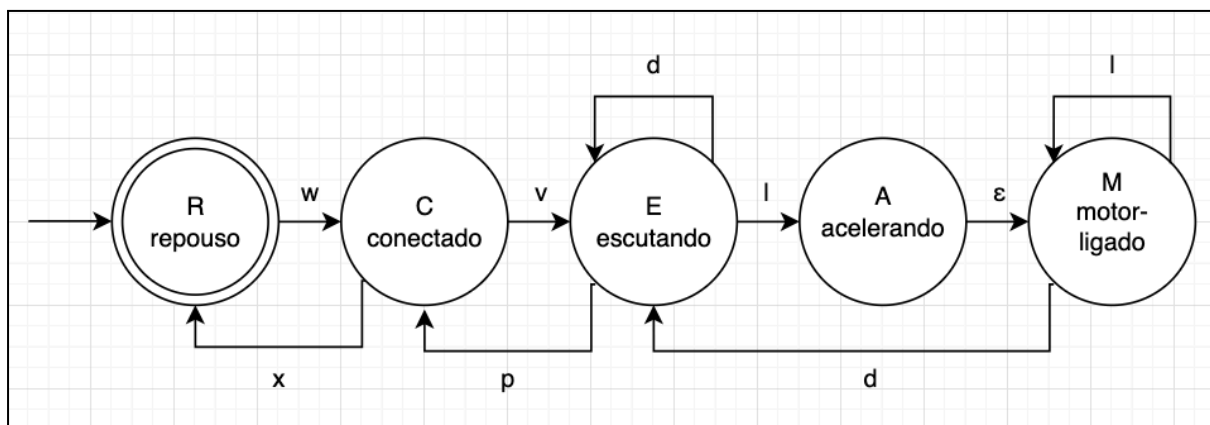


Figura 1 — Autômato da sessão de controle por voz (M1). Fonte: Autoria própria no Draw.io.

Em M1, o caminho principal é linear: $R \rightarrow C \rightarrow E \rightarrow A \rightarrow M$. Três arcos inferiores devolvem o sistema a fases anteriores (x encerra a sessão, p encerra a escuta e d desliga os motores) e dois laços, d em E e l em M, representam comandos repetidos, que não alteram o estado.

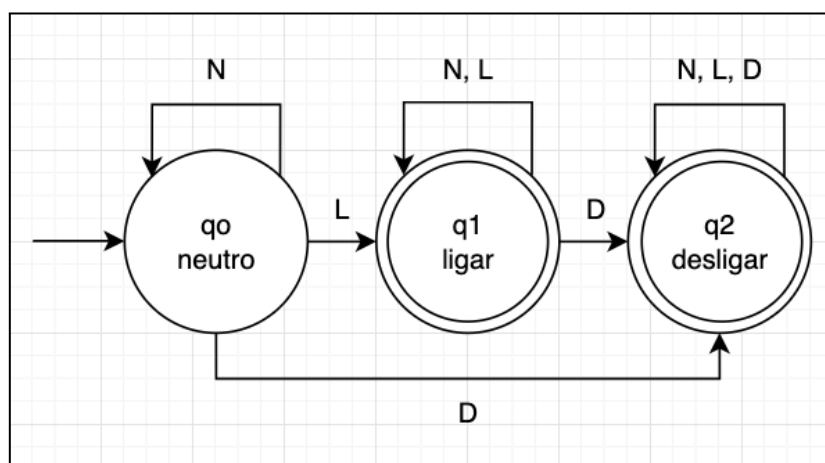


Figura 2 — Autômato reconhecedor de comandos (M2). Fonte: Autoria própria no Draw.io.

Em M2, todo caminho que sai de q_0 por L ou D chega a um estado de aceitação. O arco inferior D (de q_0 direto a q_2) e a aresta D de q_1 a q_2 mostram que uma palavra de desligar leva sempre a q_2 , de onde não se sai.

4 Tabela de transições

4.1 Função de transição de M1

A função de transição δ_1 é apresentada de forma explícita a seguir, indicando estado atual, evento de entrada e próximo estado.

Quadro 5 — Função de transição δ_1 de M1.

Estado atual	Evento	Próximo estado	Observação
R	w	C	O celular conecta à rede do ESP32 e a página é carregada.
C	v	E	O usuário ativa o microfone; começa a escuta contínua.
C	x	R	O celular sai sem ter usado a escuta; a sessão termina.
E	l	A	Frase com termo de ligar; /ligar chega ao ESP32 e a rampa começa.
E	d	E	Desligar com os motores já parados: os pinos são zerados de novo e a resposta é DESLIGADO; o estado não muda.
E	p	C	O usuário para a escuta; os motores continuam parados.
A	ϵ	M	Transição espontânea: ao fim da rampa, motoresLigados passa a verdadeiro e o servidor responde LIGADO.
M	l	M	Ligar com os motores já ligados: nada é alterado e a resposta é LIGADO.
M	d	E	Os dois motores são desligados imediatamente e a resposta é DESLIGADO.

Como uma dessas transições é ϵ , M1 é formalmente um AFN- ϵ . Os pares não listados são indefinidos e são de dois tipos. Uns não podem ocorrer: não há como ativar o microfone sem conexão, $\delta_1(R, v)$, nem dar um comando sem escuta, $\delta_1(C, l)$, e nenhum evento chega em A, porque a transição ϵ dispara antes. Outros são situações reais, mas fora do fluxo normal modelado: $\delta_1(M, p)$, $\delta_1(M, x)$ e $\delta_1(E, x)$, isto é, parar a escuta ou sair da rede com os motores ligados ou com a escuta aberta. Esses casos são discutidos na Seção 8.7.

4.2 Função de transição de M2

Quadro 6 — Função de transição δ_2 de M2.

Estado atual	Classe lida	Próximo estado	Observação
q_0	N	q_0	Palavra neutra: continua sem palavra-chave.
q_0	L	q_1	Primeira palavra de ligar.
q_0	D	q_2	Primeira palavra de desligar.
q_1	N, L	q_1	Nada muda: a frase segue sendo de ligar.
q_1	D	q_2	Uma palavra de desligar sobrepõe a de ligar.
q_2	N, L, D	q_2	Estado absorvente: a frase já é de desligar.

δ_2 é uma função total, isto é, todo par estado–classe está definido, como convém a um AFD. A prioridade do desligar aparece na última linha: em q_2 , qualquer classe leva de volta a q_2 .

4.3 Saídas associadas

As linguagens reconhecidas dizem quando as coisas acontecem, mas não o que o sistema produz. Em M1, as ações estão nas transições, à maneira de um transdutor de Mealy; em M2, a saída depende do estado em que a frase termina (Moore). O Quadro 7 reúne as saídas de M1.

Quadro 7 — Entradas lidas e saídas produzidas ao longo da sessão (M1).

Transição	Entrada lida	Saída / ação produzida
$R \rightarrow C$	GET / e GET /status	O servidor entrega a página, que exibe o estado atual dos motores (LIGADO ou DESLIGADO).
$C \rightarrow E$	Toque no botão de escuta	Reconhecimento de voz iniciado. Nenhuma requisição ao ESP32.
$E \rightarrow A$	GET /ligar	Início da rampa: o PWM dos pinos 25 e 26 cresce a cada 15 ms.
$A \rightarrow M$	— (espontânea)	Valores finais de PWM: 25 no pino 25 e 100 no pino 26; motoresLigados = verdadeiro; resposta LIGADO.
$M \rightarrow E$	GET /desligar	PWM dos dois pinos em 0; motoresLigados = falso; resposta DESLIGADO.
$E \rightarrow E$	GET /desligar	Mesma ação, sem efeito novo: pinos zerados de novo; resposta DESLIGADO.
$M \rightarrow M$	GET /ligar	Nenhuma alteração; resposta LIGADO.
$E \rightarrow C$	Toque no botão de parar	Reconhecimento encerrado. Nenhuma requisição ao ESP32.
$C \rightarrow R$	—	Nenhuma ação no ESP32: o servidor não mantém sessão.

A decisão de M2, tal como implementada na página, pode ser resumida assim:

```
frase ← transcrição.trim().toLowerCase()
se contém {desligar, desliga, apagar, off} → GET /desligar
senão, se contém {ligar, liga, acender, on} → GET /ligar
senão → "comando não mapeado" (nenhuma requisição)
```

Na primeira saída ocorre o evento d de M1; na segunda, o evento l.

Os valores de PWM (25 e 100, na escala de 0 a 255) e as respostas de texto "LIGADO" e "DESLIGADO" são saídas: não definem estados. A resposta é usada pela página para atualizar o status exibido ao usuário.

5 Linguagem do sistema

5.1 Linguagem da sessão (L_1)

Considerando $\Sigma_1 = \{w, v, l, d, p, x\}$, a linguagem L_1 reconhecida por M1 é o conjunto das seqüências de eventos que levam o sistema do estado inicial R de volta a R, isto é, zero ou mais sessões de uso completas. Cada sessão começa com a conexão (w), contém zero ou mais períodos de escuta, cada um aberto por v e fechado por p, com ciclos de ligar e desligar entre os dois, e termina com a desconexão (x).

Exemplos de seqüências válidas, aceitas por L_1 :

- ϵ — a máquina está ligada, mas nenhum celular se conectou.
- **wx** — o celular conecta e sai sem usar o microfone.
- **wvpx** — ativa e desativa o microfone sem dar nenhum comando.
- **wvldpx** — sessão mínima com lançamento: conecta, ativa o microfone, liga, desliga, para a escuta e sai.
- **wvldldpx** — comandos repetidos ("liga, liga, desliga, desliga"): ligar com os motores ligados e desligar com os motores parados não alteram o estado.
- **wvldldpx** — dois ciclos de lançamento na mesma escuta.
- **wvpvldpx** — o usuário ativa o microfone, desativa, reativa e só então comanda.
- **wvldpxwx** — duas sessões seguidas.

Exemplos de seqüências inválidas, não reconhecidas por L_1 :

- **v...** — ativar o microfone sem conexão prévia: $\delta_1(R, v)$ é indefinida.
- **wl...** — dar um comando sem microfone ativo: $\delta_1(C, l)$ é indefinida, pois sem escuta não há frase a validar.
- **wvlp** — parar a escuta com os motores ligados: $\delta_1(M, p)$ é indefinida (situação discutida na Seção 8.7).
- **wvldx** — sair da rede sem encerrar a escuta: $\delta_1(E, x)$ é indefinida.
- **wvld** — termina em E, que não é estado de aceitação: a escuta continua aberta e a sessão está incompleta.

5.2 Linguagem dos comandos (L_2)

O vocabulário de comandos é finito: *ligar*, *liga*, *acender* e *on* formam o grupo de ligar, e *desligar*, *desliga*, *apagar* e *off* formam o grupo de desligar. As seis palavras previstas inicialmente

para o produto (LIGAR, LIGA, ON, DESLIGAR, DESLIGA e OFF) estão incluídas; ACENDER e APAGAR são sinônimos presentes no código.

Sobre $\Sigma_2 = \{L, D, N\}$, a linguagem L_2 reconhecida por M2 é o conjunto das frases, vistas como sequências de classes de palavras, que contêm ao menos uma palavra-chave: $L_2 = \{ u \in \Sigma_2^* : u \text{ contém ao menos um } L \text{ ou um } D \}$. Ela se divide em duas partes, correspondentes aos dois estados de aceitação: L_{lig} , as frases com ao menos um L e nenhum D (terminam em q_1), e L_{des} , as frases com ao menos um D (terminam em q_2).

Quadro 8 — Exemplos de frases transcritas e o resultado de M2.

Frase transcrita	Sequência em Σ_2	Resultado
"liga", "ligar", "on"	L	Aceita em q_1 : /ligar.
"por favor, acender"	N N L	Aceita em q_1 : /ligar.
"desliga", "off"	D	Aceita em q_2 : /desligar.
"apagar a máquina"	D N N	Aceita em q_2 : /desligar.
"ligar e depois desligar"	L N N D	Aceita em q_2 : /desligar (o desligar tem prioridade).
"olá"	N	Rejeitada em q_0 : comando não mapeado.
"parar"	N	Rejeitada em q_0 : palavra fora do vocabulário.
"aumentar velocidade"	N N	Rejeitada em q_0 : nenhum termo-chave.

A frase "ligar e depois desligar" mostra a prioridade do desligar: ela contém uma palavra de cada grupo e é lida como desligar. A frase vazia também é rejeitada, pois não contém palavra-chave.

6 Expressão regular

Faz sentido usar expressões regulares neste projeto pelos dois lados do modelo. As duas linguagens, L_1 e L_2 , são reconhecidas por autômatos finitos e portanto são regulares, e a validação de comandos já é, na prática, uma busca de padrões em texto. As expressões a seguir são construídas em três níveis: sequências de eventos da sessão, sequências de classes de palavras e palavras formadas por caracteres.

6.1 Sessão de uso (L_1)

A linguagem L_1 , sobre $\Sigma_1 = \{w, v, l, d, p, x\}$, é expressa por:

$$L_1 = (w (v (l^* d)^* p)^* x)^*$$

Lendo de dentro para fora: $l^* d$ é um pedido de desligar precedido de zero ou mais pedidos de ligar repetidos; $(l^* d)^*$ são zero ou mais ciclos de lançamento dentro de uma mesma escuta; $v (\dots)^* p$ é um período de escuta, aberto pelo microfone e fechado quando ele é desativado; $w (\dots)^* x$ é uma conexão, com zero ou mais períodos de escuta; e o fecho externo repete sessões. Como o bloco $(l^* d)^*$ só pode terminar em d ou ser vazio, a expressão impõe que a escuta só seja encerrada com os motores desligados, que é exatamente o comportamento do fluxo normal.

6.2 Comandos numa frase (L_2)

Sobre $\Sigma_2 = \{L, D, N\}$, a linguagem L_2 e suas duas partes são:

$$L_2 = N^* (L \mid D) (L \mid D \mid N)^*$$

$$L_{\text{lig}} = N^* L (L \mid N)^*$$

$$L_{\text{des}} = (L \mid D \mid N)^* D (L \mid D \mid N)^*$$

L_2 diz que a frase pode começar com palavras neutras, mas precisa ter uma primeira palavra-chave (L ou D), depois da qual qualquer coisa é permitida. L_{lig} exige ao menos um L e proíbe D ; L_{des} exige ao menos um D , em qualquer posição.

6.3 Palavras (nível dos caracteres)

Seja Σ o conjunto dos caracteres possíveis do texto já normalizado. As classes de palavras da Seção 2.2 são definidas por:

$$R_D = \Sigma^* (\text{desligar} \mid \text{desliga} \mid \text{apagar} \mid \text{off}) \Sigma^*$$

$$R_L = \Sigma^* (\text{ligar} \mid \text{liga} \mid \text{acender} \mid \text{on}) \Sigma^*$$

Uma palavra é de classe D se pertence a R_D ; é de classe L se pertence a R_L mas não a R_D ; e é de classe N nos demais casos. Como a busca é por trecho contido na palavra, as formas longas são redundantes: toda palavra que contém "ligar" contém "liga", e toda que contém "desligar" contém "desliga". Elas foram mantidas por legibilidade, para espelhar o vocabulário do código que desenvolvemos.

7 Gramática

7.1 Gramática da sessão (G_1)

A seguir apresenta-se uma gramática regular (linear à direita) $G_1 = (V_1, \Sigma_1, P_1, R)$ que gera exatamente a linguagem L_1 , com $V_1 = \{R, C, E, M\}$ e $\Sigma_1 = \{w, v, l, d, p, x\}$:

$$\begin{aligned} R &\rightarrow w C \mid \varepsilon \\ C &\rightarrow v E \mid x R \\ E &\rightarrow l M \mid d E \mid p C \\ M &\rightarrow l M \mid d E \end{aligned}$$

Cada não terminal corresponde a um estado do autômato. R é o repouso e é o único que pode encerrar a derivação ($R \rightarrow \varepsilon$). C e E são a conexão e a escuta. M representa os estados ACELERANDO e MOTORES_LIGADOS combinados, já que a transição entre eles é espontânea e não consome símbolo de entrada. Uma derivação de $wvldpx$, por exemplo, é: $R \Rightarrow wC \Rightarrow wvE \Rightarrow wvlM \Rightarrow wvldE \Rightarrow wvldpC \Rightarrow wvldpxR \Rightarrow wvldpx$.

7.2 Gramática dos comandos (G_2)

A gramática regular $G_2 = (V_2, \Sigma_2, P_2, Q_0)$, com $V_2 = \{Q_0, Q_1, Q_2\}$ e $\Sigma_2 = \{L, D, N\}$, gera L_2 :

$$\begin{aligned} Q_0 &\rightarrow N Q_0 \mid L Q_1 \mid D Q_2 \\ Q_1 &\rightarrow N Q_1 \mid L Q_1 \mid D Q_2 \mid \varepsilon \\ Q_2 &\rightarrow N Q_2 \mid L Q_2 \mid D Q_2 \mid \varepsilon \end{aligned}$$

Os não terminais correspondem aos estados de M_2 , e só Q_1 e Q_2 , que são de aceitação, podem terminar a derivação. Q_0 não tem produção vazia: uma frase sem palavra-chave não é gerada. A frase "por favor, ligar" ($N N L$) é derivada por $Q_0 \Rightarrow N Q_0 \Rightarrow N N Q_0 \Rightarrow N N L Q_1 \Rightarrow N N L$. Já "ligar e depois desligar" ($L N N D$) é derivada por $Q_0 \Rightarrow L Q_1 \Rightarrow L N Q_1 \Rightarrow L N N Q_1 \Rightarrow L N N D Q_2 \Rightarrow L N N D$, terminando em Q_2 : a prioridade do desligar aparece na própria derivação.

7.3 Gramática do vocabulário (G_3)

Por fim, a estrutura dos comandos em si é uma gramática G_3 cujos terminais são as próprias palavras do vocabulário e cuja linguagem tem oito elementos:

CMD $\rightarrow$ LIG | DES

LIG $\rightarrow$ ligar | liga | acender | on

DES $\rightarrow$ desligar | desliga | apagar | off

CMD é o símbolo inicial e LIG e DES são as duas famílias de comando, cujos resultados são, respectivamente, os eventos l e d de M1. G_3 é regular, pois gera uma linguagem finita, e complementa G_2 : G_3 define quais palavras são comandos, e G_2 , como esses comandos aparecem numa frase.

8 Justificativa

8.1 Por que autômatos finitos?

A escolha de autômatos finitos, e não de um autômato com pilha ou de uma máquina de Turing, se justifica pela natureza do problema: nenhuma das duas camadas precisa memorizar informação de profundidade arbitrária nem reconhecer estruturas aninhadas. Em M1 basta saber em qual fase da sessão o sistema está; em M2 basta saber se, até aqui, a frase não trouxe palavra-chave, trouxe uma de ligar ou trouxe uma de desligar. Um número finito e fixo de estados captura os dois casos.

8.2 Correspondência com o código

Diferentemente de um firmware com tipo enumerado, o protótipo não declara os estados de M1 explicitamente: eles emergem da combinação de três informações, a existência de um celular conectado, a variável `isListening` da página e a variável `motoresLigados` do ESP32. O Quadro 9 mostra as combinações alcançáveis, que são exatamente os cinco estados de M1.

Quadro 9 — Estados de M1 e as variáveis do código que os determinam.

Estado	Celular conectado	isListening (página)	motoresLigados (ESP32)
REPOUSO	Não	—	falso
CONECTADO	Sim	falso	falso
ESCUTANDO	Sim	verdadeiro	falso
ACELERANDO	Sim	verdadeiro	falso (rampa em execução)
MOTORES_LIGADOS	Sim	verdadeiro	verdadeiro

Os eventos também têm correspondência direta: `w` é o carregamento da página; `v` e `p` são as duas ações do botão de escuta (`toggleVoice`); `l` e `d` são o resultado de M2 e chegam ao ESP32 como chamadas aos manipuladores `handleLigar` e `handleDesligar`. Já M2 corresponde à cadeia de decisões `if / else if` executada quando a página recebe uma frase reconhecida (`onresult`).

8.3 Duas máquinas ligadas por uma interface

Separar M1 e M2 foi uma decisão de modelagem. Os dois autômatos têm alfabetos de natureza diferente, palavras de uma frase de um lado e eventos de uma sessão do outro, e a única ligação entre eles é a saída de M2 (`l` ou `d`) tornar-se a entrada de M1. Fundi-los em um único autômato produziria um produto de estados e transições que multiplicaria a tabela sem acrescentar informação de comportamento. Manter a interface entre eles explícita também acompanha a arquitetura real: M2 executa no navegador e M1 descreve o sistema como um todo.

8.4 A fronteira entre estado e saída

Modelar cada palavra do vocabulário como um símbolo distinto de Σ_2 criaria oito ramos equivalentes, sem acrescentar comportamento, já que "liga" e "acender" levam ao mesmo estado. Por isso as palavras foram agrupadas em classes. O mesmo vale para os valores de PWM, as respostas de texto e o conteúdo da frase transcrita: são saídas ou dados que o sistema manipula, não fases em que ele se encontra.

8.5 A transição espontânea e a rampa de aceleração

A passagem de ACELERANDO para MOTORES_LIGADOS foi modelada como transição ε porque, no código, ela ocorre sem depender de nenhuma nova entrada: o manipulador de /ligar executa a rampa (51 passos de 15 ms, cerca de 0,8 s) e, ao terminar, marca os motores como ligados e responde. Manter A separado de M tem valor prático, pois A é a fase em que o sistema já foi comandado mas os motores ainda não estão em regime. Durante a rampa, o servidor não atende outras requisições; um pedido de desligar que chegue nesse intervalo só é atendido depois, o que equivale à sequência $l d (E \Rightarrow A \Rightarrow M \Rightarrow E)$. Por isso não é preciso definir $\delta_1(A, d)$.

8.6 A prioridade do desligar

O estado q_2 de M2 é absorvente, o que reproduz a ordem das verificações do código, que testa o grupo de desligar antes do de ligar. Além de necessária, porque "desligar" contém "ligar", essa ordem tem um efeito de segurança: diante de uma frase ambígua que contenha os dois grupos, o sistema escolhe desligar, o resultado menos arriscado para uma máquina que lança bolas.

9 Reflexões

O ponto que mais nos ensinou foi descobrir onde a validação acontece. A descrição inicial do projeto colocava essa etapa no ESP32, com o texto chegando até ele. Ao mapear o modelo para o código, percebemos que a validação é feita no navegador e que o ESP32 só recebe requisições já decididas. Foi a exigência de localizar cada transição com precisão que nos levou a separar o sistema em duas máquinas, e não em uma.

A segunda decisão importante foi a fronteira entre estado e saída. Ao reduzir as palavras a três classes e tratar PWM, respostas e transcrição como saídas, ficou claro que o autômato não descreve o que o sistema diz nem mede, mas em que fase ele está.

Por fim, o modelo também nos mostrou o que ele não resolve. A linguagem L_1 deixa indefinido o caso de parar a escuta com os motores ligados, e a linguagem L_2 aceita palavras que só contêm um termo-chave. O autômato diz o que é possível, não o que é bom, e essas duas lacunas podem virar itens de melhoria para as próximas etapas do projeto.

10 Conclusão

A modelagem apresentada demonstra que o ciclo de controle por voz do NockHug pode ser descrito por dois autômatos finitos: M1, de cinco estados e uma transição ϵ , que reconhece a linguagem $L_1 = (w(v(l^*d)^*p)^*x)^*$, e M2, de três estados, que reconhece a linguagem $L_2 = N^*(L \mid D)(L \mid D \mid N)^*$. Cada linguagem tem sua expressão regular e sua gramática linear à direita equivalentes.

Essa formalização rendeu duas coisas concretas ao projeto: uma especificação verificável do comportamento do protótipo, com o local exato de cada transição, e a identificação de pares (estado, evento) que o código ainda não trata. Ampliar o vocabulário de comandos alteraria M2 e o alfabeto Σ_2 , mas deve preservar a estrutura de M1, já que a sessão de uso continuaria a mesma.

11 Anexo I

Código do sistema. Última versão: 17/09/2026.

```
#include <WiFi.h>
#include <WebServer.h>

const char *ssid = "ESP32_Voz_Direta";
const char *password = "12345678";

WebServer server(80);

// Pinos dos motores (mantidos do código 2)
const int pinoMotor1 = 25;
const int pinoMotor2 = 26;
const int VELOCIDADE_ALVO = 50;

// Variável para rastrear o status e informar a página web
bool motoresLigados = false;

const char PAGE_MAIN[] = R"rawliteral(
<!DOCTYPE html>
<html lang="pt-BR">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width,
initial-scale=1.0">
  <title>Diagnostico de Voz ESP32 - Motores</title>
  <style>
    body { font-family: 'Segoe UI', Tahoma, Geneva, Verdana,
sans-serif; text-align: center; padding: 15px; background: #0f172a;
color: #f8fafc; margin: 0; }
    h2 { margin: 10px 0; color: #38bdf8; }
    .card { background: #1e293b; padding: 15px; border-radius: 10px;
max-width: 500px; margin: 0 auto 15px auto; box-shadow: 0 4px 6px
-1px rgba(0,0,0,0.3); }
    #btnMic { width: 100%; padding: 16px; font-size: 16px; border:
none; border-radius: 8px; background: #2563eb; color: white; cursor:
pointer; font-weight: bold; transition: 0.3s; }
    #btnMic.active { background: #dc2626; animation: pulse 1.2s
infinite; }
    #statusBox { font-size: 18px; margin: 10px 0; font-weight: bold;
}
    #consoleLog { background: #020617; border: 1px solid #334155;
border-radius: 8px; padding: 10px; text-align: left; font-family:
'Courier New', Courier, monospace; font-size: 12px; height: 220px;
overflow-y: scroll; color: #a5f3fc; }
    .log-info { color: #38bdf8; }
    .log-success { color: #4ade80; font-weight: bold; }
    .log-warn { color: #facc15; }
    .log-error { color: #f87171; font-weight: bold; }
    @keyframes pulse { 0% { opacity: 1; } 50% { opacity: 0.6; } 100%
{ opacity: 1; } }
  </style>

```

```

</head>
<body>
  <div class="card">
    <h2>ESP32 Voice Console - Motores</h2>
    <div id="statusBox">Status: <span id="ledStatus"
style="color:#94a3b8">DESCONHECIDO</span></div>
    <button id="btnMic" onclick="toggleVoice()">1. Iniciar Escuta de
Voz</button>
  </div>

  <div class="card">
    <div style="display:flex; justify-content:space-between;
align-items:center; margin-bottom:6px;">
      <span style="font-weight:bold; font-size:13px;
color:#94a3b8;">Terminal de Eventos do Navegador:</span>
      <button onclick="clearConsole()" style="background:#475569;
color:#fff; border:none; padding:4px 8px; border-radius:4px;
font-size:10px; cursor:pointer;">Limpar</button>
    </div>
    <div id="consoleLog"></div>
  </div>

  <script>
    const consoleBox = document.getElementById('consoleLog');
    let recognition;
    let isListening = false;

    function log(msg, type = 'info') {
      const now = new Date();
      const time = now.toTimeString().split(' ')[0] + '.' +
String(now.getMilliseconds()).padStart(3, '0');
      const div = document.createElement('div');
      div.className = 'log-' + type;
      div.innerText = '[' + time + ']' + msg;
      consoleBox.appendChild(div);
      consoleBox.scrollTop = consoleBox.scrollHeight;
    }

    function clearConsole() {
      consoleBox.innerHTML = '';
      log("Console limpo.", "info");
    }

    if ('webkitSpeechRecognition' in window || 'SpeechRecognition' in
window) {
      const SpeechRecognition = window.SpeechRecognition ||
window.webkitSpeechRecognition;
      recognition = new SpeechRecognition();
      recognition.continuous = true;
      recognition.lang = 'pt-BR';
      recognition.interimResults = false;

      recognition.onstart = function() {
        log("Microfone ATIVADO. Aguardando fala...", "success");
      };

      recognition.onaudiostart = function() {

```

```

    log("Captura de audio iniciada pelo driver.", "info");
};

recognition.onspeechstart = function() {
    log("Som de voz detectado! Processando...", "warn");
};

recognition.onresult = function(event) {
    const lastResult = event.results[event.results.length -
1][0].transcript.trim().toLowerCase();
    log("Frase reconhecida: \"" + lastResult + "\"", "success");

    if (lastResult.includes('desligar') ||
lastResult.includes('apagar') || lastResult.includes('desliga') ||
lastResult.includes('off')) {
        sendCmd('/desligar');
    } else if (lastResult.includes('ligar') ||
lastResult.includes('acender') || lastResult.includes('liga') ||
lastResult.includes('on')) {
        sendCmd('/ligar');
    } else {
        log("Comando nao mapeado. Diga 'ligar' ou 'desligar'.",
"warn");
    }
};

recognition.onerror = function(event) {
    log("ERRO NA API DE VOZ: " + event.error, "error");
};

recognition.onend = function() {
    log("Reconhecimento finalizado.", "info");
    if (isListening) {
        log("Reiniciando escuta continua...", "warn");
        try { recognition.start(); } catch(e) {}
    }
};

} else {
    log("ERRO: Navegador sem suporte a SpeechRecognition.",
"error");
}

function toggleVoice() {
    if (!recognition) return;
    const btn = document.getElementById('btnMic');
    if (!isListening) {
        try {
            recognition.start();
            isListening = true;
            btn.innerText = "2. Escutando... (Toque p/ Parar)";
            btn.classList.add('active');
            log("Iniciando requisicao de escuta...", "info");
        } catch(e) {
            log("Excecao ao iniciar: " + e.message, "error");
        }
    } else {
        isListening = false;
    }
}

```

```

        recognition.stop();
        btn.innerText = "1. Iniciar Escuta de Voz";
        btn.classList.remove('active');
        log("Escuta pausada pelo usuario.", "warn");
    }
}

function sendCmd(endpoint) {
    log("Enviando HTTP GET para: " + endpoint + " ...", "info");
    fetch(endpoint)
        .then(res => res.text())
        .then(data => {
            log("ESP32 respondeu: " + data, "success");
            if(data === "LIGADO") {
                document.getElementById('ledStatus').innerHTML = '<span
style="color:#4ade80">LIGADO</span>';
            } else {
                document.getElementById('ledStatus').innerHTML = '<span
style="color:#f87171">DESLIGADO</span>';
            }
        })
        .catch(err => {
            log("Falha ao comunicar com ESP32: " + err.message,
"error");
        });
}

window.onload = function() {
    log("Pagina carregada com sucesso.", "info");
    sendCmd('/status');
};
</script>
</body>
</html>
)rawliteral";

void handleRoot() {
    server.send(200, "text/html", PAGE_MAIN);
}

void handleLigar() {
    // Acelera suavemente se os motores estiverem parados
    if (!motoresLigados) {
        for (int velocidade = 0; velocidade <= VELOCIDADE_ALVO;
velocidade++) {
            analogWrite(pinoMotor1, velocidade / 2); // Reduz pela metade
(chega a 25)
            analogWrite(pinoMotor2, velocidade * 2); // Dobra a velocidade
(chega a 100)
            delay(15);
        }
        motoresLigados = true;
        Serial.println("[ESP32] -> Motores LIGADOS (Pino 25: 50%, Pino
26: 200%)");
    }
    server.send(200, "text/plain", "LIGADO");
}

```

```

void handleDesligar() {
    // Desliga os motores imediatamente
    analogWrite(pinoMotor1, 0);
    analogWrite(pinoMotor2, 0);
    motoresLigados = false;

    Serial.println("[ESP32] -> Motores DESLIGADOS");
    server.send(200, "text/plain", "DESLIGADO");
}

void handleStatus() {
    String estado = motoresLigados ? "LIGADO" : "DESLIGADO";
    server.send(200, "text/plain", estado);
}

void setup() {
    Serial.begin(115200);
    delay(1000);

    // Configura os pinos dos motores como saída
    pinMode(pinoMotor1, OUTPUT);
    pinMode(pinoMotor2, OUTPUT);

    // Garante que os motores iniciem desligados
    analogWrite(pinoMotor1, 0);
    analogWrite(pinoMotor2, 0);

    WiFi.softAP(ssid, password);

    server.on("/", handleRoot);
    server.on("/ligar", handleLigar);
    server.on("/desligar", handleDesligar);
    server.on("/status", handleStatus);

    server.begin();
    Serial.println("ESP32 Voice Server para Motores Atualizado.");
}

void loop() {
    // A lógica de manter a velocidade agora é gerenciada pelo estado
    // do pino (analogWrite)
    // Então o loop() só precisa lidar com as requisições web.
    server.handleClient();
}

```